

Exercise-2

PAGE NO.: _____

DATE ____/____/____

→ Big O notation :-

Big 'O' notation is a standardized notation used to describe the upper bound or worst case complexity of an algorithm's running time or space.

Best case : Target element found immediately in first iteration.

E.g. Array = [10, 20, 30, 40, 50]

target = 10

found at first comparison.

time complexity = $O(1)$.

Average Case : Target element found in the middle of data.

E.g. Array = [10, 20, 30, 40, 50]

target = 30

found in middle at $arr[2]$.

time complexity = $O(n/2) = O(n)$

Worst case : Target element found at last position.

E.g. Array = [10, 20, 30, 40, 50]

target = 50.

found at last.

time complexity = $O(n)$.

➤ Code:-

```
1  class Product {
2      int productId;
3      String productName;
4      String category;
5
6      Product(int productId, String productName, String category) {
7          this.productId = productId;
8          this.productName = productName;
9          this.category = category;
10     }
11
12     public String productInfo() {
13         return productId + " " + productName + " " + category;
14     }
15
16     @ public static int linearSearch(Product[] products, int targetId) {
17         for (int i = 0; i < products.length; i++) {
18             if (products[i].productId == targetId) {
19                 return i;
20             }
21         }
22         return -1;
23     }
```

```
25     @ public static int binarySearch(Product[] products, int targetId) {
26         int low = 0;
27         int high = products.length - 1;
28
29         while (low <= high) {
30             int mid = low + (high - low) / 2;
31
32             if (products[mid].productId == targetId) {
33                 return mid;
34             }
35
36             if (products[mid].productId < targetId) {
37                 low = mid + 1;
38             } else {
39                 high = mid - 1;
40             }
41         }
42
43         return -1;
44     }
```

```
46  ▶ public static void main(String[] args) {  
47  
48      Product[] products = {  
49          new Product(101, "Laptop", "Electronics"),  
50          new Product(102, "Phone", "Electronics"),  
51          new Product(103, "Shoes", "Fashion"),  
52          new Product(104, "Watch", "Accessories"),  
53          new Product(105, "Bag", "Fashion")  
54      };  
55  
56      int result1 = Product.linearSearch(products, 104);  
57      System.out.println("Linear Search: " + result1);  
58  
59      int result2 = Product.binarySearch(products, 104);  
60      System.out.println("Binary Search: " + result2);  
61  }  
62  }
```

Output:

```
"C:\Program Files\Java\jdk-24\bin\java.exe" --enable-p
```

```
Linear Search: 3
```

```
Binary Search: 3
```

```
Process finished with exit code 0
```

```
|
```

* Time Complexity of Linear Search:-

* Linear Search:-

- Time Complexity = $O(n)$.
- Check each product one by one.
- Does not require sorted array.
- Suitable for small data set.

* Binary Search:-

- Time Complexity = $O(\log n)$.
- Repeatedly divides search space into half.
- Require array to be sorted.
- Suitable for large dataset.

→ For this E-commerce platform "Binary search" is more suitable because:

- E-commerce platform contain thousands of ~~prode~~ product.
- It provide much faster search performance.
- Product record can easily be maintained.